

1 Home Assignment 2:

Build a Document Question Answering (Document QA) Agent

Develop an AI-powered Document QA agent that enables users to upload a document from their local system and ask questions based on its content. The agent should extract the document content, understand the user's query, and provide accurate answers using only the uploaded document as the source of information. If the requested information is not available in the document, the agent should respond appropriately instead of generating unsupported answers.

Expected Outcome: Users should be able to upload a document, ask multiple questions, and receive context-aware answers derived only from the uploaded document.

list of nodes:

- 1.Manua Trigger
- 2.Read /write files from disk
- 3.Extract from file
- 4.Basic LLM chain

2 Workflow Planning

2.1 Initial Workflow Planned

2.1.1 Architecture

Manual Trigger

↓

Read File from Disk

↓

Extract From File

↓

Basic LLM Chain

↑

Groq Chat Model

↓

Answer User

2.1.2 Challenges Identified During Implementation

Challenge	Impact
Manual Trigger executes once	Not suitable for interactive chat
Document processed for every question	Unnecessary processing and slower execution
No persistence	Extracted text lost after workflow completion
No separation of responsibilities	Harder to maintain and extend
Repeated execution required	Poor user experience

2.2 Revised Workflow

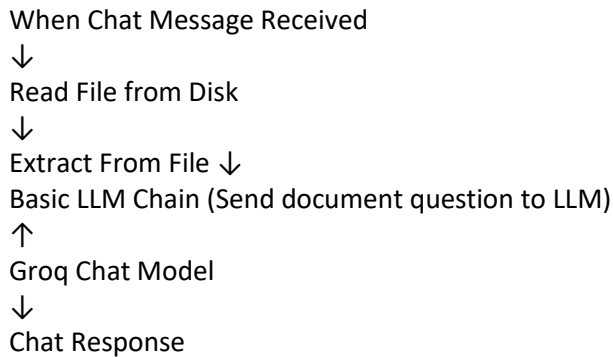
2.2.1 Workflow 1 – Document Ingestion

Manual Trigger
↓
Read File from Disk
↓
Extract From File
↓
Convert to Text File
↓
Write File to Disk

Reason: Process the document only once and store the extracted content for reuse.

Component	Role	Why it is Required
Manual Trigger	Starts the workflow manually.	Since document ingestion is a one-time activity, manual execution is sufficient. The workflow runs only when a new document is uploaded or an existing document changes.
Read File from Disk	Reads the document from the specified local file path.	Makes the document available to the workflow as binary data for further processing.
Extract From File	Extracts readable text from the document.	AI models cannot understand binary files (PDF, DOCX, etc.). This node converts the document into plain text.
Convert to File	Converts the extracted text into a text file.	The extracted content is initially JSON/text in memory. This node creates a reusable text file that can be stored.
Write File to Disk	Saves the generated text file locally.	Persists the processed document so it can be reused by the QA workflow without extracting the document again.

2.2.2 Workflow 2 – Document Question Answering



Reason: Allow users to ask multiple questions without reprocessing the document.

Component	Responsibility
Chat Trigger	Receives the user's question
Read File	Reads the stored document
Extract From File	Converts the document into plain text
Basic LLM Chain	Builds the prompt by combining the document and the user's question
Groq Chat Model	Understands the prompt and generates the answer
Chat Response	Displays the answer back to the user

2.3 3 Reason for Design Change

Why did I split it into two workflows?

The initial design used a single workflow with a Manual Trigger to process the document and answer questions. During implementation, we found that a Manual Trigger is intended for one-time execution and does not support interactive user input. Each new question required restarting the workflow, causing the document to be read and extracted repeatedly. This resulted in unnecessary processing and a poor user experience.

To address these limitations, the solution was redesigned into two workflows: a Document Ingestion workflow that processes the document once, and a Document QA workflow that uses a Chat Trigger to support multiple user questions efficiently without reprocessing the document. This modular design improves performance, maintainability, and scalability.

Challenge	Why it Happened	Impact
No way for users to ask questions interactively	Manual Trigger only starts the workflow when the Execute button is clicked.	The workflow was not conversational.
A question could not be entered dynamically	Manual Trigger does not accept user input like a Chat Trigger does.	Questions had to be hardcoded in the Basic LLM Chain prompt, which made the workflow inflexible.
Every new question required re-running the entire workflow	The workflow always started from reading and extracting the document.	Unnecessary processing and slower execution.
Document was reprocessed for every question	Reading and extracting the document were part of the same workflow as question answering.	Increased execution time and resource usage.
No separation of responsibilities	Document processing and AI interaction were tightly coupled.	Difficult to maintain, test, and extend the workflow.

3 Tasks Completed

- Workflow 1 – Document Ingestion completed.
- Workflow 2 – Document QA completed.
- Document-based question answering validated.
- Prompt engineering implemented.
- Groq Chat Model integrated.
- Workflow tested (LLM) with valid and invalid questions.

3.1 Workflow 1 – Document ingestion

- built and executed successfully.
- Document received and stored.

Document ingestion - n8n

Overview

Editor

Manual Trigger

Read/Write Files from Disk

Extract from File

Convert to File

Write file to disk

Execute workflow

Logs

Manual Trigger Success in 1ms

Success in 521ms

Manual Trigger

Read/Write Files from Disk

Extract from File

Convert to File

Write file to disk

OUTPUT

No fields - item(s) exist, but they're empty

1 item

Activate Windows

Go to Settings to activate Windows.

Document ingestion - n8n

Overview

Editor

Manual Trigger

Read/Write Files from Disk

Extract from File

Convert to File

Write file to disk

Execute workflow

Logs

Read/Write Files from Disk Success in 13ms

Success in 521ms

Manual Trigger

Read/Write Files from Disk

Extract from File

Convert to File

Write file to disk

INPUT

No fields - item(s) exist, but they're empty

1 item

OUTPUT

data

File Name: GitHub Copilot Overview for Visual Studio Code.pdf

Directory: C:/Users/dgane/.n8n-files

File Extension: pdf

Mime Type: application/pdf

File Size: 712 kB

1 item

Activate Windows

Go to Settings to activate Windows.

Document ingestion - n8n

Document ingestion - n8n

Document ingestion - n8n

localhost:5678/workflow/JZBPTihnt9bMolk

Personal / Document ingestion + Add tag

0 / 1 Publish

Star 199,768

Overview

Editor Executions Evaluations

Manual Trigger

Read/Write Files from Disk

Extract from File

Convert to File

Write file to disk

Execute workflow

Logs

Clear execution

Extract from File Success in 333ms

Input

Output

Success in 521ms

Manual Trigger

Read/Write Files from Disk

Extract from File

Convert to File

Write file to disk

Templates

Insights

Help

Settings

INPUT

data

File Name: GitHub Copilot Overview for Visual Studio Code.pdf

Directory: C:/Users/dgane/n8n-files

File Extension: pdf

Mime Type: application/pdf

File Size: 712 kB

Download

OUTPUT

Creator: Microsoft® Word 2016

CreationDate: D:20260602191121+05'30'

ModDate: D:20260602191121+05'30'

Producer: Microsoft® Word 2016

text

GitHub Copilot Overview for Visual Studio Code

GitHub Copilot is an AI-powered code completion tool that helps developers write code faster by offering intelligent suggestions. It seamlessly integrates with Visual Studio Code, providing context-aware code recommendations for a wide range of programming languages.

1. AI-Powered Suggestions

GitHub Copilot assists you in writing code by providing suggestions based on your current context. This includes:

• Completing entire functions.

• Generating repetitive code structures.

• Offering documentation for generated code.

2. Multi-Language Support

Go to Settings to activate Windows.

version: 5.4.296

Success in 521ms

Manual Trigger

Read/Write Files from Disk

Extract from File

Convert to File

Write file to disk

Templates

Insights

Help

Settings

INPUT

numpages: 4

numrender: 4

Info

PDFFormatVersion: 1.5

Language: en-IN

EncryptFilterName: [null]

IsLinearized: false

IsAcroFormPresent: false

IsXFAPresent: false

IsCollectionPresent: false

OUTPUT

data

File Name: file.txt

File Extension: txt

Mime Type: text/plain

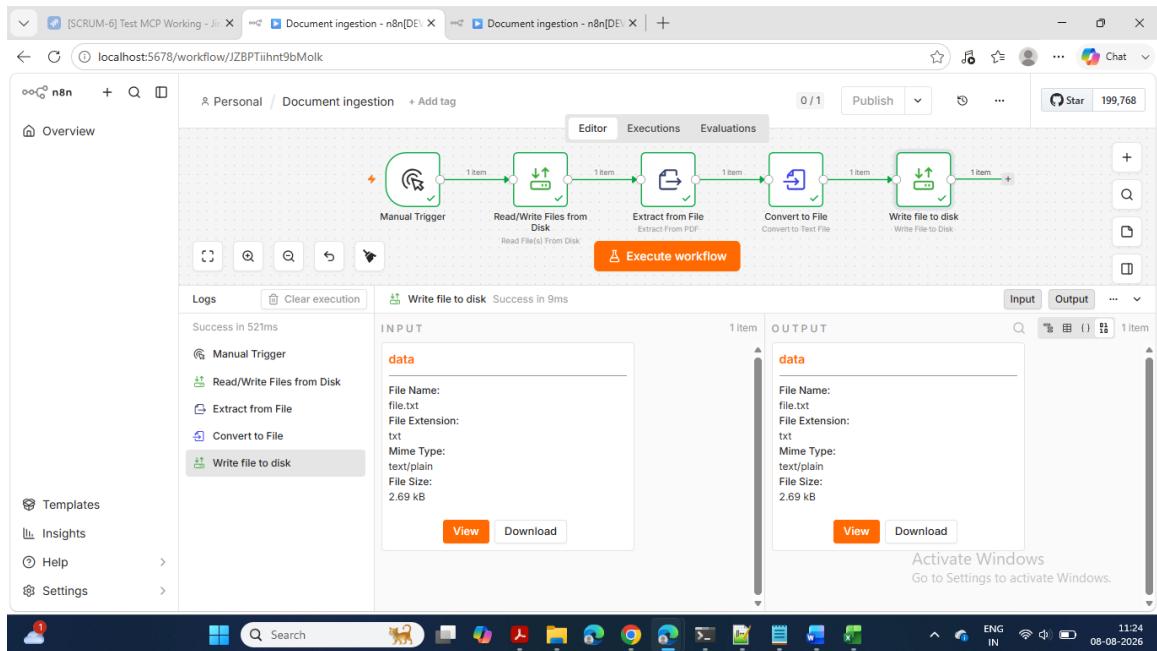
File Size: 2.69 kB

View

Download

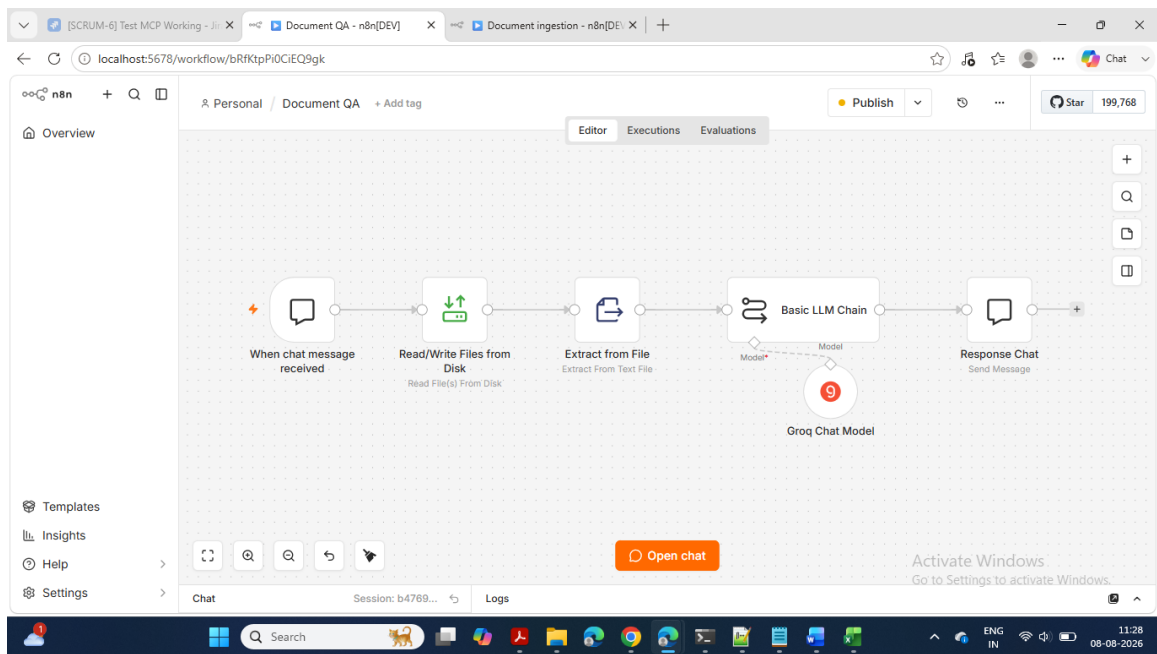
Activate Windows

Go to Settings to activate Windows.



3.2 Workflow 2 – Document QA

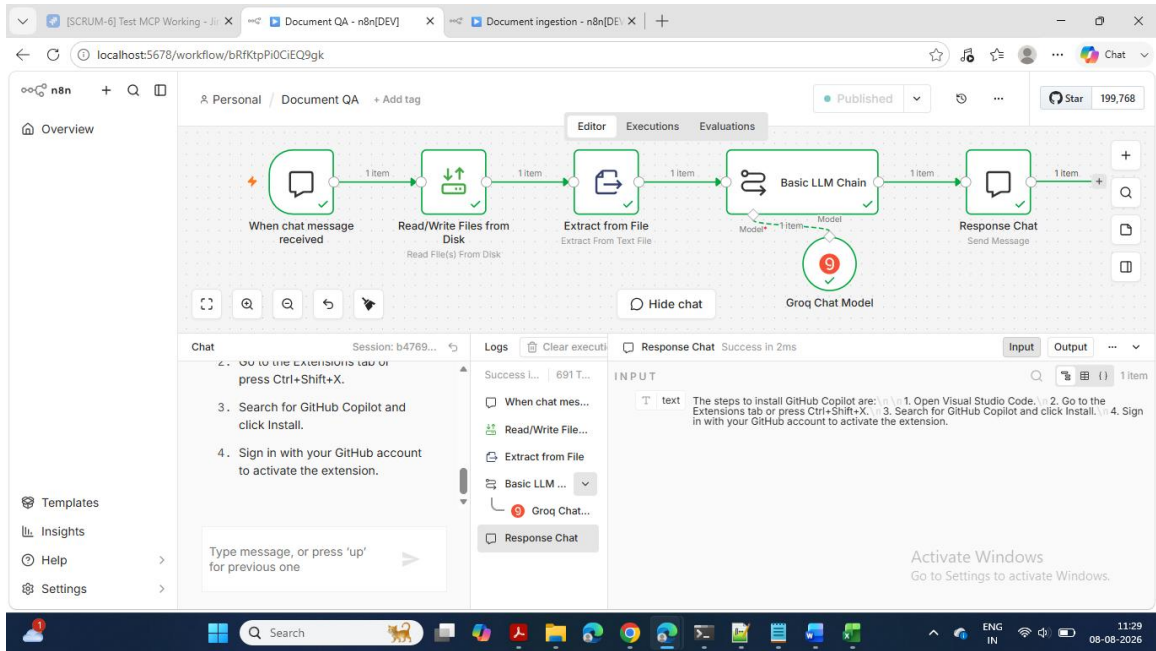
designed and executed successfully



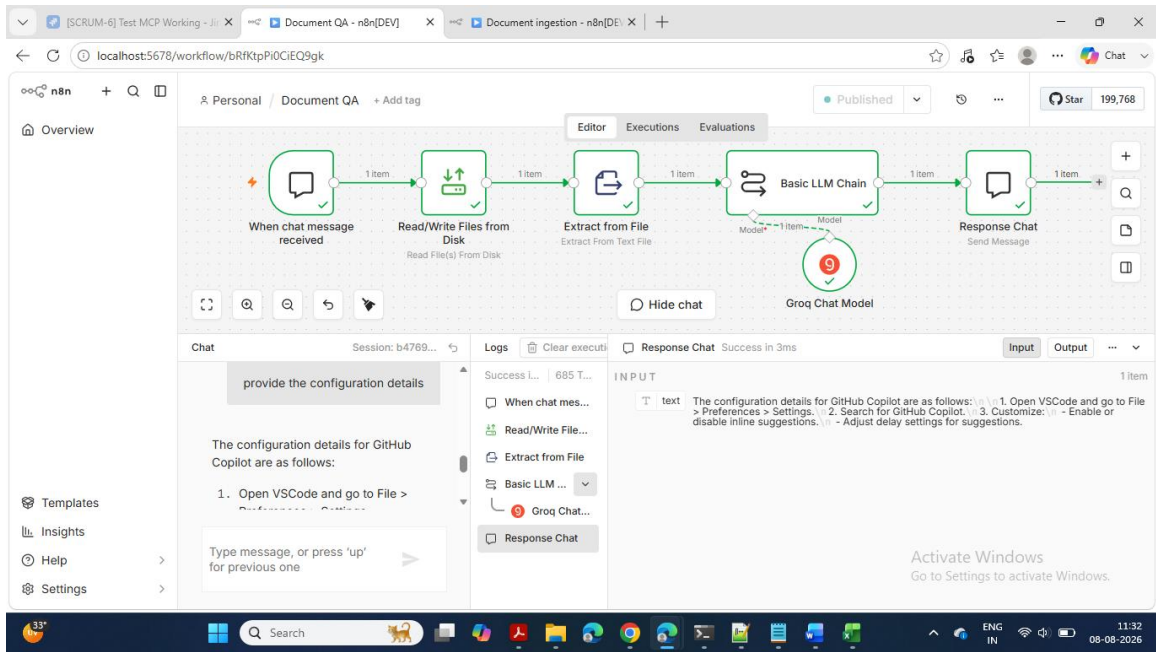
Positive testing

Testing the workflow with question based on the document

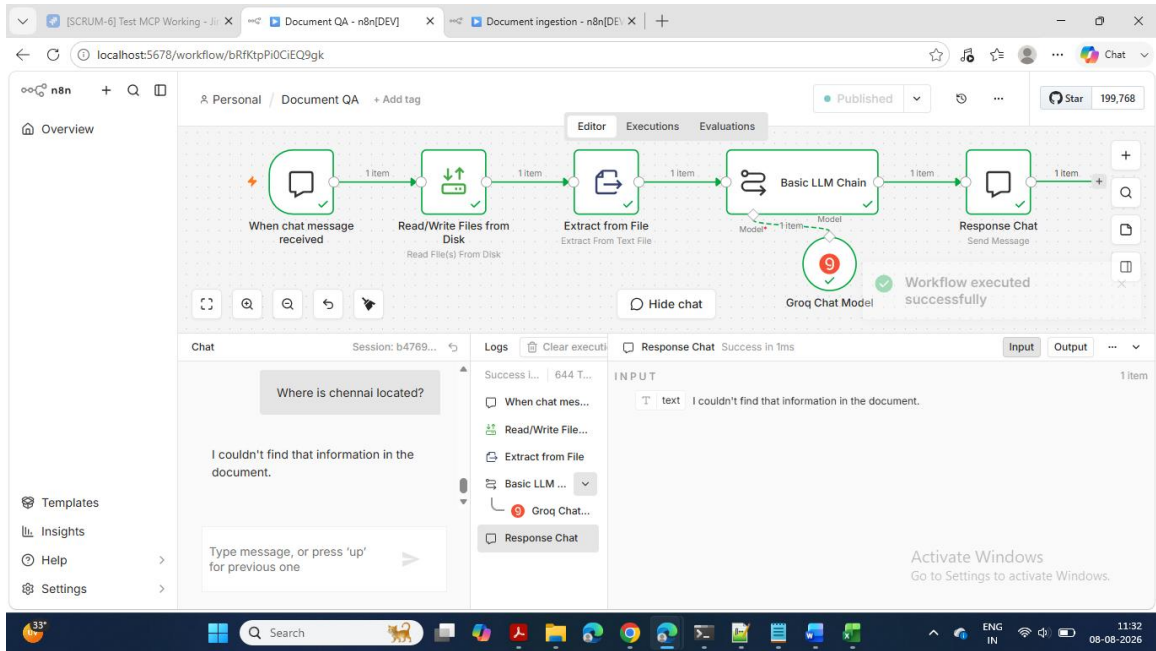
Test 1 - success



Test 2 –



Negative testing



4 Issues faced, Root Cause & Solution

4.1 Workflow 1 – Document Ingestion

Step	Issue Faced	Root Cause	Resolution
Read File from Disk	Access to the file was not allowed.	n8n file system restrictions and incorrect allowed file path configuration. In n8n v2.33.4, the Read/Write Files from Disk node has filesystem security restrictions. It only allows access to folders explicitly permitted through the <code>N8N_RESTRICT_FILE_ACCESS_TO</code> environment variable. Initially, the PDF was stored in locations such as OneDrive and <code>C:\Temp</code>, which were not allowed.	- Configured the allowed file access path in the n8n environment and used the correct local file path. Created a dedicated folder: <code>C:\Users\dgane\.n8n-files</code> - Copied the PDF into that folder. - Set the environment variable in Command Prompt: set <code>N8N_RESTRICT_FILE_ACCESS_TO=C:\Users\dgane\.n8n-files</code> - Restarted n8n from the same Command Prompt where the environment variable was set. - Updated the File(s) Selector to:

Step	Issue Faced	Root Cause	Resolution
Read File from Disk	File could not be located.	Incorrect file path or file name.	Updated the file path to the correct location and verified file availability.
Extract From File	Node showed "Node not executed".	Previous node had not been executed, so no input was available.	Used Execute Previous Nodes before testing the current node.
Convert to File	No output file was generated.	Incorrect input field mapping.	Configured the correct text input field (text) before execution.
Write File to Disk	File was not written successfully.	Incorrect binary field name or output file path.	Set the correct binary field (data) and verified the destination path.

4.2 Workflow 2 – Document Question Answering

Step	Issue Faced	Root Cause	Resolution
Chat Trigger	User questions could not be captured dynamically in the initial design.	Manual Trigger does not support conversational input.	Introduced the Chat Trigger to receive user questions.
Basic LLM Chain	LLM returned the template instead of an answer.	Prompt expressions were not configured correctly.	Corrected the prompt expressions and ensured document and question values were passed properly.
Basic LLM Chain	LLM answered using general knowledge	Prompt did not explicitly restrict the model to the document content.	Updated the prompt to instruct the LLM to answer only from the provided document and return a fallback message if the answer was unavailable.

Step	Issue Faced	Root Cause	Resolution
	instead of the document.		
Chat Response	Message was not displayed in the chat window.	Response node was not configured with the LLM output.	Mapped the Basic LLM Chain output (text) to the Chat Response node.
Workflow Execution	Connection rejected or inconsistent execution during testing.	Workflow configuration changes were not reflected in the current execution.	Saved the workflow and executed a fresh chat session after configuration updates.

5 Key Learnings

5.1 Workflow 1 – Document Ingestion

Technical Learnings

- Understood the purpose of each node involved in document ingestion.
- Learned how to read files from the local file system using **Read File from Disk**.
- Learned how to extract readable text from documents using **Extract From File**.
- Understood the difference between **Binary Data** and **JSON/Text Data** in n8n.
- Learned how to convert extracted text into a reusable text file.
- Learned how to store processed files locally using **Write File to Disk**.
- Understood how n8n handles file operations and local file access restrictions.

5.2 Workflow 2 – Document Question Answering

- Understood the role of an **LLM (Groq Chat Model)** in generating responses.
- Learned how **Basic LLM Chain** prepares the prompt for the LLM.
- Understood the concept of **context injection** by providing document content to the LLM.
- Learned how prompt engineering influences the quality and accuracy of responses.
- Understood how to reduce hallucinations by restricting the model to document content.
- Learned how to build an interactive chat workflow using **Chat Trigger**.
- Understood the difference between **Manual Trigger** and **Chat Trigger**.
- Learned how to pass data between different nodes using expressions.
- Understood the use of:
 - `$json`
 - Previous node references (`$( 'Node Name' )`)
- Learned how to return AI-generated responses back to the chat interface.
- Understood why a **single workflow** was not suitable for continuous user interaction.
- Learned the importance of separating document processing from question answering.
- Understood the benefits of modular workflow design.
- Learned how reusable workflows improve maintainability and scalability.
- Designing solutions that can be extended to **Retrieval-Augmented Generation (RAG)** in the future.

Problem-Solving Learnings

- Learned how to troubleshoot file access issues.
- Understood how to identify incorrect field mappings.
- Learned to validate node outputs before moving to the next node.
- Developed a systematic approach to debugging workflow execution.